

Expense Report Tracker

A Serverless Approval Workflow on AWS

Module	5ENTAPP / E5WMD — Enterprise Software Engineering
Superviseur	Dr. Abdelhak TOUITI
Membres	[NOM1], [NOM2], [NOM3], [NOM4]
Dépôt GitHub	https://github.com/[votre-repo]

1. Introduction

Dans la plupart des entreprises, la gestion des notes de frais repose encore sur des échanges d'emails, des pièces jointes envoyées par messagerie instantanée, ou des fichiers Excel transmis manuellement au service financier. Ce fonctionnement crée des problèmes concrets : des demandes perdues dans les boîtes mail, un risque de double remboursement faute de suivi centralisé, des audits trimestriels chronophages, et des seuils de validation contournés car aucun mécanisme automatisé ne les impose.

Notre projet répond à ce besoin en proposant une application serverless de gestion de notes de frais, construite sur AWS. Deux rôles sont définis : l'employé, qui crée et soumet ses notes de frais avec un justificatif photographique, et le responsable financier, qui examine chaque soumission, consulte le reçu, puis approuve ou rejette la demande avec un commentaire. L'ensemble du workflow suit une machine à états dont les transitions sont validées côté serveur, rendant impossible toute manipulation depuis le client.

L'objectif pédagogique est de maîtriser la conception d'une architecture serverless complète : authentification avec Cognito, stockage NoSQL avec DynamoDB, gestion sécurisée de fichiers avec S3, logique métier dans Lambda, et interface cross-platform avec .NET MAUI.

2. Architecture

L'architecture repose entièrement sur des services managés AWS, sans aucun serveur à administrer. Chaque service a été choisi pour un rôle précis, et les interactions entre eux sont sécurisées par IAM selon le principe du moindre privilège.

2.1 Justification des services

Service	Rôle	Justification
.NET MAUI	Interface utilisateur	Cross-platform (iOS, Android, Windows). Deux vues distinctes selon le rôle authentifié.
Cognito	Authentification	Gestion des utilisateurs, groupes (employees/finance), et émission de JWT contenant les claims.
API Gateway	Point d'entrée REST	Routage des requêtes vers Lambda, avec un authorizer Cognito qui valide chaque JWT.
Lambda (.NET 8)	Logique métier	5 fonctions : création, listing, détail, mise à jour de statut, et génération d'URL pré-signée.
DynamoDB	Stockage des données	Table unique avec PK/SK composite et deux GSIs pour supporter différents patterns de requêtes.
S3	Stockage des reçus	Bucket privé, accès exclusivement par URL pré-signées générées par Lambda.
IAM	Permissions	Rôle Lambda avec accès DynamoDB (CRUD) et S3 (PutObject/GetObject) uniquement.

2.2 Flux de données

Le client MAUI s'authentifie via Cognito et reçoit un JWT. Ce token est envoyé dans le header Authorization de chaque requête à l'API Gateway. L'authorizer Cognito valide le token avant de transmettre la requête à la Lambda correspondante. Chaque Lambda extrait le userId et le groupe (cognito:groups) directement depuis les claims JWT, sans appel supplémentaire à Cognito. Les Lambda accèdent à DynamoDB pour les données et à S3 pour les reçus, avec des permissions IAM strictement limitées.

Pour l'upload de reçus, le client demande d'abord une URL pré-signée à Lambda (POST /presigned-url), puis upload directement le fichier vers S3 via cette URL. Ce pattern évite de faire transiter le fichier binaire par API Gateway et Lambda, qui ont des limites de taille de payload.

Le diagramme d'architecture annoté est fourni en annexe (architecture-diagram.svg).

3. Modèle de données DynamoDB

3.1 Table principale : ExpenseReports

On utilise une table unique avec un design single-table. La clé de partition (PK) est de la forme USER#<userId> et la clé de tri (SK) est EXPENSE#<ulid>. Ce choix permet de requêter toutes les dépenses d'un employé en une seule opération Query sur la PK, ce qui est le pattern d'accès le plus fréquent.

Attribut	Type	Exemple
PK (Partition Key)	String	USER#abc-123-def
SK (Sort Key)	String	EXPENSE#01HXYZ...
ExpenseId	String	01HXYZ... (ULID)
UserId	String	abc-123-def (Cognito sub)
UserEmail	String	employee@company.com
Amount	Number	340.00
Category	String	meals
Description	String	Déjeuner client
Status	String	Submitted
ReceiptKey	String	receipts/abc/01HX.../receipt.jpg
ReviewerComment	String	Approuvé, conforme
CreatedAt	String	2026-05-10T14:30:00Z

3.2 Global Secondary Indexes (GSIs)

GSI	Partition Key	Sort Key	Usage
StatusIndex	Status	CreatedAt	Le responsable financier liste toutes les dépenses en statut Submitted, triées par date de création.
CategoryIndex	Category	CreatedAt	Reporting : filtrer les dépenses par catégorie (travel, meals, equipment, o...)

3.3 Patterns d'accès

Pattern 1 — L'employé liste ses notes de frais : Query sur PK = USER#<userId>. Retourne toutes les dépenses de cet utilisateur, triées par ULID (donc chronologiquement).

Pattern 2 — Le financier consulte les demandes en attente : Query sur le GSI StatusIndex avec PK = Submitted. Retourne toutes les dépenses en attente d'approbation, triées par date de création (les plus anciennes d'abord pour un traitement FIFO).

Pattern 3 — Récupérer une dépense spécifique : GetItem avec PK + SK. O(1), pas de scan.

Pattern 4 — Reporting par catégorie : Query sur le GSI CategoryIndex. Permet au CFO de voir toutes les dépenses « travel » du trimestre par exemple.

4. Implémentation des éléments clés

4.1 Machine à états (State Machine)

La machine à états est le cœur du système. Elle garantit qu'une expense ne peut passer que par des transitions valides, quelles que soient les requêtes envoyées par le client. Les transitions autorisées sont définies dans un dictionnaire côté Lambda :

```
Draft → Submitted (employé) | Submitted → Approved (finance) | Submitted →  
Rejected (finance, commentaire obligatoire) | Rejected → Resubmitted (employé,  
propriétaire uniquement) | Resubmitted → Submitted (employé)
```

Avant chaque transition, la Lambda vérifie trois choses : (1) le rôle de l'utilisateur via les claims JWT — seul un membre du groupe « finance » peut approuver ou rejeter ; (2) la propriété de l'expense — un employé ne peut modifier que ses propres notes ; (3) la validité de la transition — le dictionnaire retourne le nouveau statut ou une erreur 409 Conflict. De plus, un mécanisme d'optimistic locking via ConditionExpression empêche les race conditions : si le statut a changé entre la lecture et l'écriture, DynamoDB rejette l'update.

4.2 Contrôle d'accès basé sur les rôles (RBAC)

Le RBAC fonctionne à deux niveaux. D'abord, l'API Gateway valide le JWT via l'authorizer Cognito — toute requête sans token valide est rejetée avec un 401. Ensuite, chaque Lambda extrait les claims du JWT (userId via « sub », rôle via « cognito:groups ») et applique les règles métier.

Extrait du code (UpdateExpenseStatus/Function.cs) :

```
var groups = request.RequestContext.Authorizer.Claims["cognito:groups"];  
var isFinance = groups.Contains("finance");  
  
if (FinanceActions.Contains(action) && !isFinance)  
    return Response(403, "Only finance managers can approve/reject.");
```

4.3 Flux des URLs pré-signées (Pre-signed URLs)

Le bucket S3 est entièrement privé : aucun objet n'est accessible publiquement. Pour uploader un reçu, le client appelle POST /presigned-url avec l'expenseId et le nom du fichier. Lambda génère une URL PUT signée (valide 15 minutes) et la retourne au client. Le client upload alors le fichier directement vers S3, sans passer par API Gateway. Pour afficher un reçu, le même endpoint génère une URL GET signée (valide 60 minutes).

La clé S3 suit la convention receipts/<userId>/<expenseId>/<fileName>, ce qui permet de retrouver facilement les reçus et d'éviter les collisions de noms.

4.4 Infrastructure as Code (CloudFormation)

L'ensemble de l'infrastructure est définie dans un template CloudFormation unique (scripts/cloudformation.yaml). Ce template crée tous les services (Cognito, DynamoDB, S3, Lambda, API Gateway, IAM) en une seule commande. Les Outputs exportent les identifiants nécessaires à la configuration du client MAUI (API URL, User Pool ID, Client ID).

5. Conclusion

5.1 Difficultés rencontrées

La principale difficulté a été la conception du modèle DynamoDB. Contrairement à une base relationnelle où l'on peut ajouter des jointures a posteriori, DynamoDB impose de connaître ses patterns d'accès dès le départ. Il a fallu anticiper les besoins du financier (lister par statut) et du reporting (filtrer par catégorie) pour définir les GSIs appropriés. Le choix d'un design single-table avec PK/SK composite s'est avéré plus naturel qu'une approche multi-tables.

La gestion des pre-signed URLs a aussi demandé de la réflexion. Il fallait trouver le bon équilibre entre la durée de validité (trop courte = l'upload échoue sur les connexions lentes, trop longue = risque de sécurité) et le bon content-type pour que le navigateur MAUI interprète correctement les images.

5.2 Solutions adoptées

Pour la machine à états, on a opté pour un dictionnaire de transitions plutôt qu'un Step Functions, car le workflow est suffisamment simple pour être géré en mémoire dans une seule Lambda. L'ajout d'un ConditionExpression sur le statut actuel agit comme un verrou optimiste et empêche les race conditions sans avoir besoin de transactions DynamoDB.

Pour le RBAC, plutôt que de gérer les rôles dans DynamoDB, on s'appuie directement sur les groupes Cognito intégrés au JWT. Cela simplifie l'architecture : pas de table de rôles à maintenir, pas de synchronisation entre Cognito et la base de données.

5.3 Améliorations possibles

Plusieurs pistes d'amélioration sont envisageables pour une version de production. D'abord, l'ajout de notifications (Amazon SNS ou SES) pour alerter l'employé lorsque sa note de frais est approuvée ou rejetée. Ensuite, l'intégration d'un seuil de double validation automatique : toute dépense supérieure à 500€ pourrait nécessiter deux approbations distinctes, en utilisant AWS Step Functions pour orchestrer le workflow multi-étapes.

Côté données, un système de TTL sur DynamoDB pourrait archiver automatiquement les dépenses approuvées après un an. Enfin, l'ajout d'un dashboard analytique avec Amazon QuickSight permettrait au CFO de visualiser les tendances de dépenses en temps réel, sans requêtes manuelles.

Ce projet nous a permis de comprendre concrètement les avantages et les contraintes d'une architecture serverless. La courbe d'apprentissage est réelle, notamment sur DynamoDB et IAM, mais le résultat est une application scalable, sécurisée et déployable en quelques minutes via CloudFormation.